

# Window: A Fast-Paced, Swipe-Based E-Commerce Application

Project Proposal for UCS503P  
Submitted to: Miss Paramveer Kaur  
Thapar Institute of Engineering and Technology

Piyush Malik, CSED\*      Paarth Singh, CSED<sup>†</sup>      Bhuvik Garg, CSED<sup>‡</sup>

August 13, 2026

## 1 Higher-order goal

*... secondary framing*

This project aims to revolutionize the way businesses—particularly those selling clothes, shoes, wardrobe items, paintings, decorations, and groceries—interact with their customers. By leveraging a fast-paced, swipe-based discovery interface similar to dating apps, we aim to increase customer engagement and facilitate direct interactions between buyers and sellers. The platform will be built using Tauri to ensure a fast, cross-platform desktop experience.

## 2 Time-to-value

*... primary heuristic*

- We will deliver meaningful increments quickly by prototyping the core swipe-based product discovery and chat modules first, validating the interaction loop with mock data.
- We will prioritize fast feedback by iterating weekly and ensuring the matching and communication pipelines are tested early.
- Success is defined by what can be delivered and measured in the first iteration (a working swipe and chat loop), followed by subsequent additions like availability updates and reviews.

## 3 Problem Statement

Currently, e-commerce platforms can be overwhelming for users due to cluttered interfaces and lack of direct communication with sellers. Businesses face challenges in engaging customers effectively. Common pain points include:

- **Decision Fatigue:** Traditional grids and lists of products lead to choice overload, making it hard for customers to make quick decisions.
- **Impersonal Experience:** Buyers lack a direct line of communication with sellers, making negotiations or specific inquiries difficult.

---

\*Roll No: 1024030167, Email: pmalik.be24@thapar.edu

<sup>†</sup>Roll No: 1024030160, Email: psingh20.be24@thapar.edu

<sup>‡</sup>Roll No: 1024030550, Email: bgarg.be24@thapar.edu

- **Static Listings:** Product availability is often not updated in real-time, leading to canceled orders and dissatisfaction.
- **Lack of Engagement:** Standard catalogs fail to capture the user’s attention in a fast-paced digital environment.

**Why this matters now:** With diminishing attention spans, a gamified, fast-paced shopping experience can significantly boost user retention. Furthermore, fostering direct communication helps businesses build stronger customer relationships.

## 4 Proposed Solution

*... Software Proposition*

### 4.1 Overview

Build a **cross-platform application** using **Tauri**, called **Window**, that provides a swipe-based e-commerce platform for businesses and buyers:

- **Swipe Interface:** A fast-paced discovery feed where users left/right swipe on products to show interest.
- **Flexible Pricing:** Businesses can upload products without mandatory prices to encourage direct inquiries and negotiations, increasing interaction.
- **Real-time Chat:** Direct communication channel between buyer and seller initiated upon showing interest.
- **Live Availability:** Frequent and automated updates on product availability directly communicated to interested buyers.
- **Review System:** Buyers can leave reviews and ratings for both the products and the sellers.

### 4.2 Core workflow

1. Business owners onboard and upload their product catalogs (with or without prices) onto the platform.
2. Buyers use the swipe interface to discover products, swiping right to show interest or left to skip.
3. A right swipe triggers a connection, opening a chat between the buyer and the seller for inquiries, negotiation, and finalizing the purchase.
4. The system continuously tracks and updates the buyer on the availability of their liked products.
5. Post-interaction, buyers can rate and review the product and the seller.

### 4.3 Operational constraints for an educational, course-project setting

- Ensure the application is lightweight and performs optimally on desktop via Tauri.
- Focus on the core mechanics of swipe discovery and real-time chat rather than complex payment gateways.
- Design for deployability and ease of use, using managed backend services.

## 5 Solution Approach

*... Engineering focus*

This project focuses on building a seamless, high-performance desktop application using modern web technologies wrapped in Tauri.

### 5.1 Core Mechanics

*... non-research*

The system will rely on standard implementation patterns:

- **Swipe Logic:** Implemented via standard frontend gestures and state management, without needing complex recommendation algorithms initially.
- **Real-time Chat:** Powered by WebSockets or standard real-time database integrations (e.g., Firebase, Supabase) to ensure instant message delivery.
- **Availability Tracking:** Simple event-driven updates from the seller dashboard pushed to the buyer's notification queue.

### 5.2 Architecture

*... Tauri-based solution*

A modern, decoupled architecture:

- **Frontend (Tauri):** A responsive and performant UI built with a modern web framework (e.g., React, Vue, or Svelte) wrapped in Tauri to run natively on desktop operating systems.
- **Backend API:** A centralized API handling authentication, product feeds, user profiles, and the chat engine.
- **Database:** A fast, scalable database capable of handling real-time status updates and messaging payloads.

### 5.3 CI/CD and fast delivery enabler

CI/CD pipelines will ensure:

- Automated builds of the Tauri application across target operating systems.
- Execution of unit tests for both frontend logic and backend services.
- Rapid iteration cycles to refine the swiping and chat functionalities based on testing feedback.

## 6 Evaluation Criterion

*... measurable and attributable*

We define success by the fluidity of the user interaction and the reliability of the communication channels.

### 6.1 Primary evaluation metric

**Interaction Latency:** The time taken from a user swiping right on a product to the initiation of a chat session or interest logging.

- Target: Median interaction latency  $\leq 2$  seconds.
- Attribution: Measured via client-side telemetry and database timestamp logging.

## 6.2 Secondary evaluation metrics

- **Message Delivery Time:** The time taken for a chat message to be delivered from buyer to seller (Target:  $\leq 1$  second).
- **Availability Sync Accuracy:** The delay between a seller marking a product out-of-stock and the buyer's UI reflecting the change.
- **User Engagement:** The average number of swipes per active session during pilot testing.

## 6.3 Pilot validation plan

*... high-level*

- Mock a catalog with 50-100 diverse products across categories (clothes, shoes, groceries, etc.).
- Recruit small groups of testers to act as buyers and mock businesses, evaluating the discoverability and chat experience.

## 7 Scalability

*... with foresight*

1. **Scalable (theoretically):** The system should support high concurrent user loads by:
  - Decoupling the product feed generation from the real-time chat service.
  - Caching product catalogs effectively.
  - Employing scalable WebSocket architectures for the chat functionality.
2. **Operationally deployable (practically):**
  - The Tauri app can be easily distributed as standalone binaries.
  - The backend will run on cloud infrastructure with auto-scaling capabilities for the chat servers.

## 8 Engine availability heuristic

A rich ecosystem of tools is available to implement this:

- **Tauri** for the desktop application wrapper (combining Rust backend with web frontend).
- Modern frontend frameworks (React/Vue) for swipe animations and interactive UI.
- Real-time backend platforms (e.g., Supabase, Firebase) for chat and live availability updates.
- Cloud storage solutions for product images.
- Automated build tools like GitHub Actions for cross-platform Tauri compilation.

We will leverage these existing technologies to focus on user experience and workflow rather than foundational infrastructure.

## 9 Project scope and deliverables

*... iteration-friendly*

### 9.1 Initial deliverable

*... first iteration*

- Seller dashboard to upload products (image, description, optional price).
- Buyer application (Tauri-based) featuring the swipe interface (right = interest, left = pass).
- Core interest-logging mechanism saving matched products to the buyer's profile.

## 9.2 Subsequent deliverables

- Real-time chat module for buyer-seller communication on 'matched' products.
- Live availability updates and push notifications.
- Review and rating system for sellers and specific products.
- Polished animations and enhanced UI/UX elements.

## 10 Risks and mitigations

- **Spam and Quality Control:** Unpriced products might lead to spam inquiries. Mitigation: Limit daily swipes or require basic verification for buyers.
- **Performance on low-end devices:** Swipe animations can be resource-intensive. Mitigation: Optimize frontend code and leverage Tauri's lightweight nature over Electron.
- **Real-time Infrastructure Complexity:** Managing WebSockets can be challenging. Mitigation: Use established BaaS providers (like Supabase) to handle real-time sync robustly.
- **App Distribution:** Distributing desktop apps requires code signing. Mitigation: Use generic builds for testing or rely on GitHub Actions for automated basic builds.

## 11 Summary

Window is a proposed Tauri-based e-commerce platform designed to bring the fast-paced, engaging experience of swipe-based discovery to retail businesses. By omitting mandatory pricing to foster communication, integrating real-time chat, and providing live availability updates, the platform aims to create a highly interactive and personalized shopping experience. The project emphasizes rapid iteration, focusing on interaction latency and reliable communication as key success metrics, utilizing a modern decoupled architecture suitable for an educational environment.